

Estrutura de repositórios

Contexto

Preparar e definir estrutura de repositórios para iniciar o desenvolvimento da primeira API da LATAM Platform V2, estabelecendo padrões de versionamento, branching e qualidade de código.

Decisões Tomadas

1. Estratégia de Repositório: Multi-repo

Decisão: Utilizar **multi-repositório** (um repositório por aplicação/produto).

Justificativa:

- Maior autonomia entre times e produtos
- Produtos evoluem em paralelo sem conflitos de dependência
- Falta de experiência prática da equipe com monorepos
- Evitar complexidade de gerenciamento de monorepo neste momento

2. Estrutura de Repositórios

```
github.com/latam-platform/  
├─ financial-api/           ← API de Conciliação e Settlement (.NET)  
├─ financial-front/        ← Frontend de Conciliação e Settlement (Next.js)  
└─ infrastructure/         ← Docker Compose + IaC (Terraform/CDK) + DB configs
```

Princípio de Nomenclatura:

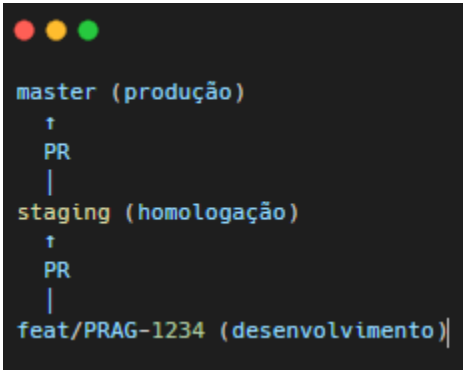
- **APIs:** {produto}-api (ex: conciliation-api, settlement-api)
- **Frontends:** {produto}-front (ex: conciliation-front, settlement-front)
- **Libs:** Nome descritivo (ex: core)
- **Infra:** infrastructure

Separação por Responsabilidade:

Tipo	Tecnologia	Exemplos	Objetivo
API	.NET 8	conciliation-api, settlement-api	Backend e lógica de negócio
Frontend	Next.js	conciliation-front, settlement-front	Interface do usuário
Core	.NET 8	core	Biblioteca compartilhada (conexões DB, helpers)
Infra	Docker/Terraform	infrastructure	IaC, compose, configurações locais

3. Estratégia de Branching

Modelo definido: Simplificado com master + staging + feature branches



Fluxo de Trabalho

Features:

```
# 1. Criar branch a partir de master
git checkout master
git pull
git checkout -b feat/PRAG-1234

# 2. Desenvolver e commitar
git commit -m "feat(conciliation): add transaction validation"

# 3. Push e abrir PR para staging
git push origin feat/PRAG-1234
# → Abrir PR: feat/PRAG-1234 → staging

# 4. Após aprovação e merge em staging, abrir PR para master
# → Abrir PR: feat/PRAG-1234 → master

# 5. Após merge em master, deletar a branch
git branch -d feat/PRAG-1234
git push origin --delete feat/PRAG-1234
```

Hotfixes:

```
# 1. Criar branch a partir de master
git checkout master
git pull
git checkout -b hotfix/PRAG-5678

# 2. Corrigir e commitar
git commit -m "fix(settlement): correct payment calculation"

# 3. PR direto para master (urgência)
git push origin hotfix/PRAG-5678
# → Abrir PR: hotfix/PRAG-5678 → master

# 4. Após merge, deletar a branch
```

Proteções de Branch

Branch	Proteção	Commits Diretos	PR Obrigatório	Aprovadores
master	Protegida	Bloqueado	Obrigatório	1 qualquer
staging	Protegida	Bloqueado	Obrigatório	1 qualquer
feat/*	Não	Permitido	-	-

hotfix/*	Não	Permitido	-	-
----------	-----	-----------	---	---

4. Nomenclatura de Branches

Padrão estabelecido:

```
feat/[CARD-ID]      → Features normais
hotfix/[CARD-ID]    → Correções urgentes
```

Exemplos:

- feat/PRAG-1234 - Implementar validação de transações
- hotfix/PRAG-5678 - Corrigir cálculo de pagamento

Importante:

- Usar sempre o ID do card do Jira/Azure DevOps
- Manter lowercase
- Separar com hífen

5. Padrão de Commits: Conventional Commits

Formato:

```
<type>(<scope>): <description>

[optional body]

[optional footer]
```

Types permitidos:

Type	Uso	Exemplo
feat	Nova funcionalidade	feat(conciliation): add transaction import
fix	Correção de bug	fix(settlement): correct payment calculation
docs	Documentação	docs(readme): update setup instructions
refactor	Refatoração sem mudança de comportamento	refactor(core): simplify connection factory
test	Adicionar/modificar testes	test(settlement): add unit tests for payment service
chore	Tarefas de manutenção	chore(deps): update entity framework to 8.0
perf	Melhoria de performance	perf(conciliation): optimize query for large datasets
ci	Mudanças em CI/CD	ci(pipeline): add sonarqube analysis

Scopes sugeridos:

- conciliation - Módulo de conciliação
- settlement - Módulo de settlement
- core - Biblioteca core
- infra - Infraestrutura
- api - Camada de API
- domain - Camada de domínio
- db - Database/migrations

Exemplos completos:

```

# Feature
git commit -m "feat(conciliation): add CSV file upload endpoint"

# Bug fix
git commit -m "fix(settlement): prevent duplicate payment processing"

# Refactoring
git commit -m "refactor(core): extract connection string resolver to separate class"

# Documentation
git commit -m "docs(api): add swagger documentation for transaction endpoints"

# Breaking change
git commit -m "feat(settlement)!: change payment status enum values

BREAKING CHANGE: payment status values changed from numeric to string"

```

6. Política de Code Review

Definição:

Aspecto	Decisão
PR obrigatório?	Sim, para master e staging
Aprovadores necessários	1 aprovador qualquer
Aprovador específico?	Não (neste momento)
Auto-merge?	Não
Deletar branch após merge?	Sim (automático)

Checklist de PR (sugestão):

```

## Descrição
<!-- Breve descrição da mudança -->

## Tipo de mudança
- [ ] Bug fix
- [ ] Nova feature
- [ ] Breaking change
- [ ] Documentação

## Checklist
- [ ] Código segue padrões do projeto
- [ ] Testes unitários adicionados/atualizados
- [ ] Documentação atualizada
- [ ] Sem warnings de build
- [ ] Migration criada (se aplicável)

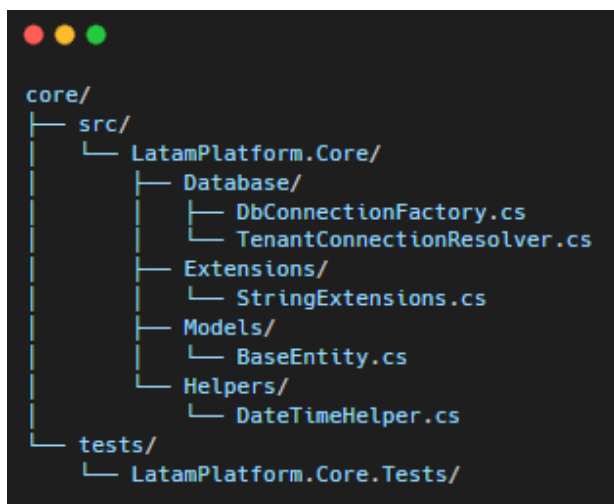
## Card relacionado
PRAG-XXXX

```

7. Repositório Core (Biblioteca Compartilhada)

Propósito:

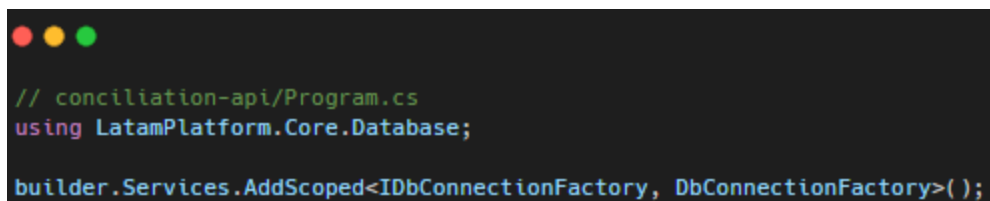
- Código compartilhado entre APIs
- Conexões com banco de dados
- Utilitários comuns
- Modelos de domínio compartilhados (se aplicável)



Distribuição:

- NuGet package interno (recomendado)
- Ou referência direta via projeto local (desenvolvimento)

Exemplo de uso:



```
// conciliation-api/Program.cs
using LatamPlatform.Core.Database;

builder.Services.AddScoped<IDbConnectionFactory, DbConnectionFactory>();
```

The code snippet shows a C# file named 'Program.cs' in the 'conciliation-api' directory. It includes the 'using LatamPlatform.Core.Database;' statement and a line of code to register the database connection factory with the service container: 'builder.Services.AddScoped<IDbConnectionFactory, DbConnectionFactory>();'.